TRD — шпаргалка команды

Разделы: 1. Что искать по условию · 2. Чек-лист · 3. Математика → C++ · 4. Типы и переполнение · 5. Подключения, ввод и вывод · 6. STL · 7. Математика и числа · 8. Префиксы, два указателя, бинпоиск · 9. Длинная арифметика · 10. Динамика · 11. Строки (и склейка, числа из строк) · 12. Геометрия · 13. Графы · 14. Перебор и жадность · **Продвинутое:** 15. Комбинаторика · 16. Строки-2 · 17. Структуры данных · 18. Графы-2 · 19. Ещё динамика · 20. Геометрия-2 · **21. Python: что можно использовать**

1. Что искать по условию

Ключевые слова → раздел

В условии	Скорее всего	Раздел
«сколько способов», «минимальная стоимость»	динамика	10
«кратчайший путь», «за сколько шагов», лабиринт	BFS / Дейкстра	13
«связаны ли», «сколько групп / островов»	DFS / DSU	13
много запросов «сумма на отрезке»	префиксные суммы	8
«самый длинный отрезок, где ...»	два указателя	8
«минимальное X, при котором можно ...»	бинпоиск по ответу	8
«по модулю 10^9+7», простые, делители, НОД	математика	7
число на 20+ цифр	длинная арифметика	9
точки, площади, окружности, отрезки	геометрия	12
«максимальный выигрыш», раздать / упорядочить	сортировка + жадность	14
«сколькими способами выбрать», C из n по k	комбинаторика	15
найти подстроку, вхождения, сравнить куски строки	КМП / Z / хеш	16
много запросов + элементы МЕНЯЮТСЯ	Фенвик / дерево отрезков	17
зависимости, «что делать сначала», порядок	топосорт	18
пути между всеми парами / отрицательные веса	Флойд / Форд-Беллман	18
соединить всё дорогами минимальной цены	каркас (Краскал)	18
разбить на пары «кто с кем»	паросочетание (Кун)	18

2. Чек-лист перед отправкой

Проверь	Где ошибались (задача)
Прогнал **пример из условия**, а не свои числа?	«Дороги»
Формат вывода буква в букву: порядок, регистр, без лишних слов?	«Статистика», «Арбузы»
Произведения, площади, большие суммы — в **long long**?	«Пятью пять», «Дачники»
Счётчики = 0 **до** цикла? Мин/макс стартуют с a[0], а не с 0?	«Статистика», «Быки и коровы», «Арбузы»
push_back того, что **прочитал**, а не n?	«Домашнее задание», «Сортировка времени»
cout ответа **после** цикла, а не внутри?	«Быки и коровы», «Дороги»
cin >> a >> b >> c — без запятых? Сначала прочитал, потом посчитал?	«Две окружности»
&& и ||: «далеко ИЛИ внутри» — это ||. Сравниваю одно с одним (квадрат с квадратом)?	«Две окружности»
Крайние случаи: n = 1, 0, отрицательные, все равны?	«Сумма»
Несколько тестов (t)? Ответ на каждый, массивы — внутри while?	Codeforces 1129

3. Математика → C++

В условии	В C++	Внимание
|x|	`abs(x)`	<cstdlib> для int и long long (или llabs); для double — fabs(x) из <cmath>
a ≤ x ≤ b	`a <= x && x <= b`	a <= x <= b компилируется, но считает НЕВЕРНО
x = y (сравнить)	`x == y`	одно = — это присвоить
и / или / не	`&& || !`	and / or / not тоже работают
x равно a и b	`x == a && x == b`	не x == a && b
x ∈ {1, 2, 3}	`x == 1 || x == 2 || x == 3`	каждый раз пишется заново «x == ...»
x делится на k	`x % k == 0`	
x чётное / нечётное	`x % 2 == 0 / x % 2 != 0`	у отрицательных x % 2 == -1, поэтому не «== 1»
x^2	`x * x`	x ^ 2 — это XOR, НЕ степень!
x^n	`powll(x, n)`	раздел 15; pow() даёт double и ошибается на больших числах
2^n	`1LL << n`	для n до 62
√x	`sqrt(x)`	double; сравнивать лучше квадраты: a*a <= b
⌈a / b⌉ (вверх)	`(a + b - 1) / b`	для a, b > 0
a / b целое	`a / b`	7 / 2 = 3; у отрицательных округляет к нулю: -7 / 2 = -3
a / b дробью	`(double)a / b`	7.0 / 2 = 3.5
остаток	`a % b`	-7 % 2 = -1: у отрицательных остаток с минусом
остаток ≥ 0	`((a % b) + b) % b`	нужен, когда a может быть отрицательным
max, min	`max(a, b), min(a, b), max({a, b, c})`	фигурные скобки для трёх и больше
НОД, НОК	`gcd(a, b), lcm(a, b)`	<numeric>, C++17
сумма цифр	`while (n > 0) { s += n % 10; n /= 10; }`	s = 0 до цикла
последняя цифра / без неё	`n % 10 / n / 10`	
сколько цифр в числе	`to_string(n).size()`	
1 + 2 + ... + n	`n * (n + 1) / 2`	в long long
n!	`fact(n)`	раздел 15; влезает в long long до n = 20
C(n, k)	`nCk(n, k)`	раздел 15
вниз / вверх / обычное округление	`floor(x) / ceil(x) / round(x)`	результат double; (int)x отбрасывает дробь
символ-цифра ↔ число	`c - '0' / char('0' + d)`	'7' - '0' = 7
сравнить строки	`s < t`	по словарю; "9" > "10" — числа-строки сравнивай сначала по длине
«выражение = 0»	`if (expr == 0)`	не expr = 0
символ / строка	`'-' / "-"`	одинарные — символ, двойные — строка
прочитать три числа	`cin >> a >> b >> c`	не cin >> a, b, c
последний элемент	`v.back()` или `v[n - 1]`	не v[-1] (это Python)
«очень большое» число	`INT_MAX / LLONG_MAX`	<climits>; самое маленькое — INT_MIN

4. Типы и переполнение

Тип	Диапазон	Когда
int	$\pm 2.1 \cdot 10^9$	счётчики, индексы, числа до 10^9
long long	$\pm 9.2 \cdot 10^{18}$	произведения, площади, суммы — почти всегда
double	~15 значащих цифр	только дроби; сравнивать fabs(a - b) < 1e-9
строка	любая длина	больше 10^{18} → длинная арифметика (раздел 9)

• a * b, где a и b до 10^9, не влезает в int — даже в сравнении. Пиши 1LL * a * b.
• accumulate(v.begin(), v.end(), **0LL**) — с 0 сумма считается в int.
• Сдвиг больше 30: 1LL << 40. Координаты в геометрии — сразу long long, не float.

5. Подключения, ввод и вывод

Какая библиотека для чего и что в ней есть

Библиотека	Для чего	Функции и что они делают
<iostream>	ввод и вывод	**cin** — читать; **cout** — писать; **cerr** — вывод для отладки (в ответ не идёт)
<iomanip>	оформление вывода	**setprecision(k)** — сколько знаков; **fixed** — считать знаки после точки; **setw(n)** — ширина вывода; **setfill(c)** — чем заполнять пустое место
<sstream>	строка как поток	**stringstream ss(s)** — разбирать строку: ss >> a >> w достаёт числа и слова; **getline(ss, item, ',')** — по разделителю
<cstdio>	файлы	**freopen** — читать из файла и писать в файл вместо экрана
<vector>	растущий массив	**push_back** — добавить в конец; **pop_back** — убрать последний; **size** — сколько; **back, front** — последний, первый; **empty** — пустой ли; **clear** — очистить; **insert, erase** — вставить, удалить; **resize** — изменить размер (раздел 6)
<string>	строки	**to_string** — число в строку; **stoi, stoll, stod** — строку в число; **substr** — кусок; **find** — найти кусок; **append** — приклеить; **size** — длина
<algorithm>	алгоритмы над массивом	**sort** — сортировка; **reverse** — перевернуть; **max_element, min_element** — где максимум, минимум; **count** — сколько раз встречается; **find** — найти; **lower_bound, upper_bound, binary_search** — бинарный поиск (массив отсортирован); **unique** — убрать соседние повторы; **next_permutation** — следующая перестановка; **swap** — поменять местами; **min, max** — меньшее, большее (min({a, b, c}) — из трёх)
<numeric>	ряд чисел	**accumulate** — сумма; **gcd, lcm** — НОД, НОК; **iota** — заполнить 0 1 2 ...; **partial_sum** — префиксные суммы
<cmath>	дробная математика	**sqrt** — корень; **pow** — степень (получается дробь!); **floor, ceil, round** — вниз, вверх, ближайшее; **fabs** — модуль дроби; **hypot** — длина вектора; **atan2** — угол вектора; **sin, cos** — в радианах
<climits>	границы типов	**INT_MAX, INT_MIN, LLONG_MAX** — «очень большое» число, чтобы искать минимум
<cstdlib>	целые числа	**abs, llabs** — модуль int и long long
<cctype>	символы	**isdigit, isalpha** — цифра ли, буква ли; **toupper, tolower** — в заглавную, в строчную
<set> <map> <unordered_map>	множество и словарь	**set** — без повторов, всегда по порядку; **map** — ключ → значение; **unordered_map** — то же без порядка, но быстрее
<stack> <queue> <deque>	стопка, очередь, дек	**stack** — последний пришёл, первый ушёл; **queue** — первый пришёл, первый ушёл (нужна для BFS); **priority_queue** — куча, достаёт максимум (для Дейкстры); **deque** — добавлять с обоих концов
<bitset>	набор битов	**bitset<8>** — 8 битов; **.count()** — сколько единиц
<functional>	функции как значения	**greater<int>()** — сортировка по убыванию и куча-минимум
<utility>	пары	**pair** — пара значений (first, second); **swap** — поменять местами

```
#include <iostream>       #include <iomanip>        #include <sstream>
#include <vector>         #include <string>         #include <algorithm>
#include <numeric>        #include <cmath>          #include <climits>
#include <cstdlib>        #include <cstdio>         #include <cctype>
#include <set>            #include <map>           #include <unordered_map>
#include <stack>          #include <queue>          #include <deque>
#include <bitset>         #include <functional>     #include <utility>
using namespace std;
using ll = long long;
```

В условии разрешены: STL (vector, algorithm, string и др.), assert.h, ctype.h, io.h, iomanip.h, ios.h, iostream.h, istream.h, limits.h, locale.h, math.h, memory.h, minmax.h, ostream.h, search.h, stdio.h, stdlib.h, string.h. Строка **#include <bits/stdc++.h>** в этом списке не названа: на GCC она обычно работает, но безопаснее подключать по отдельности. <sstream> и <unordered_map> прямо не названы — если сомневаешься, спроси судей.

Ввод и вывод: что делает каждая функция и когда она нужна

Код	Для чего	Когда нужно и подвох
`cin >> x;`	прочитать одно значение (число или слово)	пропускает пробелы и переносы строк перед значением
`cin >> a >> b >> c;`	прочитать несколько значений подряд	неважно, в одной они строке или в разных
`string s; cin >> s;`	прочитать одно слово	читает до первого пробела; слова с пробелом внутри так не прочитать
`getline(cin, line);`	прочитать всю строку целиком, с пробелами	имена, предложения; читает до конца строки (до Enter)
`cin >> ws;`	пропустить пробелы и Enter	ставь между **cin >> n** и **getline**. Без него getline прочитает пустой остаток первой строки
`while (cin >> x) { ... }`	читать, пока данные не кончатся	когда в условии не сказано, сколько чисел
`vector<string> f(n);` `for (auto &row : f) cin >> row;`	прочитать поле из символов	f[i][j] — символ в строке i, столбце j; пробелов внутри строк поля быть не должно
`ios::sync_with_stdio(false);` `cin.tie(nullptr);`	ускорить ввод и вывод	первые две строки main(); при 10^5 и больше чисел без них бывает TLE. После них не смешивай cin / cout с scanf / printf
`cout << x << '\n';`	вывести значение и перейти на новую строку	'\n' быстрее endl: endl каждый раз ещё и сбрасывает буфер, в большом цикле это медленно
`cout << a << ' ' << b << '\n';`	вывести через пробел	пробел и перенос — это символы, пиши в одинарных кавычках
`cout << (x > 0 ? "YES" : "NO");`	выбрать одно из двух слов	запись «условие ? если да : если нет». Регистр букв — как в условии
`cout << fixed << setprecision(6) << x;`	вывести дробь с 6 знаками после точки	fixed значит «считай знаки после точки». Без fixed setprecision(3) оставит 3 значащие цифры: 3.14159 → 3.14, а не 3.142. Нужен <iomanip>
`cout << setw(5) << x;`	вывести число с шириной 5 (выровнять)	действует только на ближайшее значение; setfill('0') — чем заполнять пустое место, остаётся до отмены
`cerr << x << '\n';`	вывести значение для отладки	в ответ не попадает (другой поток). Перед отправкой лучше убрать
`freopen("input.txt", "r", stdin);` `freopen("output.txt", "w", stdout);`	читать из файла и писать в файл вместо экрана	только если в условии сказано про input.txt и output.txt. Нужен <cstdio>
`stringstream ss(line);` `ss >> a >> w;`	разобрать прочитанную строку на слова и числа	сначала getline, потом разбор; while (ss >> w) берёт по одному слову. Нужен <sstream>

6. STL — справочник

v — вектор. **v.begin()** — его начало, **v.end()** — место ПОСЛЕ последнего элемента. sort(v.begin(), v.end()) читается «отсортируй от начала до конца».

VECTOR — массив, который растёт

создать из значений	`vector<int> v = {3, 1, 2};`
n элементов, все нули	`vector<int> z(n, 0);`
таблица n на m из нулей	`vector<vector<int>> g(n, vector<int>(m, 0));`
добавить x в конец	`v.push_back(x);`
убрать последний	`v.pop_back();`
сколько элементов	`v.size();`
последний элемент (не v[-1]!)	`v.back();`
первый элемент	`v.front();`
пустой ли	`v.empty();`
очистить	`v.clear();`
вставить x на позицию 1	`v.insert(v.begin() + 1, x);`
удалить элемент с позиции 1	`v.erase(v.begin() + 1);`
прочитать весь вектор	`for (auto &e : v) cin >> e;`
вывести весь вектор	`for (int e : v) cout << e << ' ';`

STRING — строка

создать	`string s = "hello";`
длина	`s.size();`
первый символ; s.back() — последний	`s[0];`
приклеить в конец	`s += "!";`
кусок: с позиции 1, длина 3	`s.substr(1, 3);`
позиция куска; string::npos, если не нашли	`s.find("ll");`
перевернуть	`reverse(s.begin(), s.end());`
число -> строка	`string t = to_string(123);`
строка -> int	`int num = stoi("456");`
строка -> long long	`ll big = stoll("9000000000");`
символ-цифра -> число ('7' -> 7)	`int d = ch - '0';`
число -> символ-цифра (7 -> '7')	`char r = '0' + d;`
цифра ли / буква ли	`isdigit(ch); isalpha(ch);`
в заглавную / в строчную	`toupper(ch); tolower(ch);`

СОРТИРОВКА И ПОИСК (v = вектор, диапазон: от v.begin() до v.end())

по возрастанию. (Пузырька в библиотеке нет — есть sort)	`sort(v.begin(), v.end());`
по убыванию	`sort(v.begin(), v.end(), greater<int>());`
своё правило: p раньше q?	`sort(v.begin(), v.end(), [](int p, int q) { return p < q; });`
перевернуть порядок	`reverse(v.begin(), v.end());`
максимум (звёздочка = значение)	`int mx = *max_element(v.begin(), v.end());`
ИНДЕКС максимума	`int imx = max_element(v.begin(), v.end()) - v.begin();`
сколько раз встречается x	`int cnt = count(v.begin(), v.end(), x);`
есть ли x	`bool has = find(v.begin(), v.end(), x) != v.end();`
убрать повторы (сначала sort!)	`v.erase(unique(v.begin(), v.end()), v.end());`
сумма (0LL, а не 0!)	`ll sum = accumulate(v.begin(), v.end(), 0LL);`
максимум из трёх; min({a, b, c}) — минимум	`int m3 = max({a, b, c});`
поменять местами	`swap(a, b);`

БИНАРНЫЙ ПОИСК (вектор уже ОТСОРТИРОВАН по возрастанию)

есть ли x	`bool found = binary_search(v.begin(), v.end(), x);`
индекс первого элемента >= x	`int lb = lower_bound(v.begin(), v.end(), x) - v.begin();`
индекс первого элемента > x	`int ub = upper_bound(v.begin(), v.end(), x) - v.begin();`
сколько раз x встречается	`int times = ub - lb;`

ДРУГИЕ КОНТЕЙНЕРЫ

множество: без повторов, всегда по возрастанию	`set<int> st;`
добавить / удалить / есть ли (0 или 1)	`st.insert(x); st.erase(x); st.count(x);`
словарь: ключ -> значение	`map<string, int> mp;`
счётчик; mp.count(k) — есть ли ключ	`mp["key"]++;`
пара; pr.first, pr.second	`pair<int, int> pr = {a, b};`
top() = МАКСИМУМ	`priority_queue<int> pqMax;`
top() = МИНИМУМ	`priority_queue<int, vector<int>, greater<int>> pqMin;`
очередь: первый пришёл — первый ушёл	`queue<int> q; q.push(x); q.front(); q.pop();`
стопка: последний пришёл — первый ушёл	`stack<int> sk; sk.push(x); sk.top(); sk.pop();`
с обоих концов	`deque<int> dq; dq.push_front(x); dq.push_back(x);`

7. Математика и числа

Функция	Что делает	Пример
isPrime(n)	простое ли число	isPrime(97) = 1, isPrime(91) = 0 (91 = 7·13)
sieve(n)	решето: все простые до n сразу	sieve(30): 2 3 5 7 11 13 17 19 23 29
divisors(n)	все делители по возрастанию	divisors(36): 1 2 3 4 6 9 12 18 36
factorize(n)	разложение на простые	$360 = 2^3 \cdot 3^2 \cdot 5$
myGcd(a, b), myLcm(a, b)	НОД, НОК (то же, что gcd, lcm)	НОД(48, 18) = 6, НОК(4, 6) = 12
binpow(a, b, m)	a в степени b по модулю m, быстро	binpow(2, 10, m) = 1024
digitSum(n)	сумма цифр	digitSum(12345) = 15
toBase(n, base)	число в другую систему счисления	toBase(255, 16) = FF, toBase(10, 2) = 1010
stoll(s, nullptr, base)	из другой системы в обычную	stoll("FF", nullptr, 16) = 255

```
// до корня: делители ходят парами
bool isPrime(ll n) {
    if (n < 2) return false;
    for (ll i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
// все простые до n (решето)
vector<bool> sieve(int n) {
    vector<bool> pr(n + 1, true);
    pr[0] = false;
    if (n >= 1) pr[1] = false;
    for (int i = 2; (ll)i * i <= n; i++)
        if (pr[i])
            for (int j = i * i; j <= n; j += i) pr[j] = false;
    return pr;
}
```

```
// все делители по возрастанию
vector<ll> divisors(ll n) {
    vector<ll> d;
    for (ll i = 1; i * i <= n; i++)
        if (n % i == 0) {
            d.push_back(i);
            if (i != n / i) d.push_back(n / i);
        }
    sort(d.begin(), d.end());
    return d;
}
// Евклид: НОД(a,b) = НОД(b, a%b)
ll myGcd(ll a, ll b) {
    while (b != 0) { ll t = a % b; a = b; b = t; }
    return a;
}
// сначала делим!
ll myLcm(ll a, ll b) { return a / myGcd(a, b) * b; }
```

```
const ll MOD = 1000000007;
// a^b % m за log(b)
ll binpow(ll a, ll b, ll m) {
    ll res = 1;
    a %= m;
    while (b > 0) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
// по модулю: (a+b)%M   (a*b)%M в ll   ((a-b)%M+M)%M   a/b -> a*binpow(b,M-2,M)%M
```

```
ll digitSum(ll n) { ll s = 0; while (n > 0) { s += n % 10; n /= 10; } return s; }
// a, b > 0
ll ceilDiv(ll a, ll b) { return (a + b - 1) / b; }
```

```
// 10 -> base (2..16)
string toBase(ll n, int base) {
    if (n == 0) return "0";
    string dig = "0123456789ABCDEF", r;
    while (n > 0) { r += dig[n % base]; n /= base; }
    reverse(r.begin(), r.end());
    return r;
}
// base -> 10:  stoll("FF", nullptr, 16) == 255
```

8. Префиксы, два указателя, бинпоиск

Префиксные суммы — что это

p[i] — сумма первых i чисел массива. Для a = 3 1 4 1 5 получается p = 0 3 4 8 9 14. Сумма на отрезке от l до r (включительно) = **p[r + 1] − p[l]**. Например, a[1..3] = 1 + 4 + 1 = p[4] − p[1] = 9 − 3 = 6. p считается один раз за n шагов, дальше любой вопрос «сумма на отрезке» — за один шаг. Без префиксов на каждый вопрос пришлось бы суммировать циклом — для 10^5 вопросов это долго.

Два указателя: левый и правый край отрезка идут по массиву только вперёд, каждый — один раз.
Бинпоиск: каждый шаг отбрасывает половину вариантов.

```
// ПРЕФИКС-СУММЫ: сумма [l, r] (с 0, включительно) = p[r+1] - p[l]
vector<ll> buildPref(const vector<ll> &a) {
    vector<ll> p(a.size() + 1, 0);
    for (int i = 0; i < (int)a.size(); i++) p[i + 1] = p[i] + a[i];
    return p;
}
```

```
// 2D: P[i+1][j+1] = g[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]
// прямоугольник (r1,c1)-(r2,c2): P[r2+1][c2+1]-P[r1][c2+1]-P[r2+1][c1]+P[r1][c1]
vector<vector<ll>> buildPref2D(const vector<vector<ll>> &g) {
    int R = g.size(), C = g[0].size();
    vector<vector<ll>> P(R + 1, vector<ll>(C + 1, 0));
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            P[i + 1][j + 1] = g[i][j] + P[i][j + 1] + P[i + 1][j] - P[i][j];
    return P;
}
```

```
// ДВА УКАЗАТЕЛЯ: самый длинный отрезок с суммой <= S (числа >= 0)
int longestAtMost(const vector<ll> &a, ll S) {
    int l = 0, best = 0;
    ll cur = 0;
    for (int r = 0; r < (int)a.size(); r++) {
        cur += a[r];
        while (cur > S) cur -= a[l++];
        best = max(best, r - l + 1);
    }
    return best;
}
```

```
// БИНПОИСК РУКАМИ: первый индекс, где a[i] >= x (массив отсортирован)
int firstGE(const vector<ll> &a, ll x) {
    // ответ в [lo, hi], hi = "нет такого"
    int lo = 0, hi = a.size();
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        // подходит -> левее (mid тоже кандидат)
        if (a[mid] >= x) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

```
// БИНПОИСК ПО ОТВЕТУ: минимальное t, при котором ok(t) == true
// ok монотонна: false false ... false true true ... true
// ok(hi) обязано быть true
ll minTrue(ll lo, ll hi, function<bool(ll)> ok) {
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

9. Длинная арифметика (числа как строки, столбиком)

В C++ нет встроенных больших чисел: самое большое — long long ($\approx 9 \cdot 10^{18}$). Большие числа хранят строкой и считают столбиком, как в школе: с последней цифры, с переносом. Названия addBig, mulBig — это функции из этого раздела, не библиотека. (В Python большие числа встроены — см. раздел 21.)

```
// a + b
string addBig(const string &a, const string &b) {
    string res;
    int i = (int)a.size() - 1, j = (int)b.size() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int s = carry;
        if (i >= 0) s += a[i--] - '0';
        if (j >= 0) s += b[j--] - '0';
        res += char('0' + s % 10);
        carry = s / 10;
    }
    reverse(res.begin(), res.end());
    return res;
}

// a - b, a >= b
string subBig(const string &a, const string &b) {
    string res;
    int i = (int)a.size() - 1, j = (int)b.size() - 1, borrow = 0;
    while (i >= 0) {
        int d = (a[i] - '0') - borrow - (j >= 0 ? b[j] - '0' : 0);
        borrow = 0;
        if (d < 0) { d += 10; borrow = 1; }
        res += char('0' + d);
        i--; j--;
    }
    while (res.size() > 1 && res.back() == '0') res.pop_back();
    reverse(res.begin(), res.end());
    return res;
}

// длинное * k (k <= 1e9)
string mulSmall(const string &a, ll k) {
    if (k == 0 || a == "0") return "0";
    string res;
    ll carry = 0;
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        ll cur = (a[i] - '0') * k + carry;
        res += char('0' + cur % 10);
        carry = cur / 10;
    }
    while (carry > 0) { res += char('0' + carry % 10); carry /= 10; }
    reverse(res.begin(), res.end());
    return res;
}

// длинное * длинное
string mulBig(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<int> r(n + m, 0);
    for (int i = n - 1; i >= 0; i--)
        for (int j = m - 1; j >= 0; j--) {
            int cur = (a[i] - '0') * (b[j] - '0') + r[i + j + 1];
            r[i + j + 1] = cur % 10;
            r[i + j] += cur / 10;
        }
    string res;
    for (int d : r) if (!(res.empty() && d == 0)) res += char('0' + d);
    return res.empty() ? "0" : res;
}

// a < b ? (№7!)
bool lessBig(const string &a, const string &b) {
    // СНАЧАЛА длина
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}
```

10. Динамика

Четыре вопроса: что хранит dp[i] (словами)? Чему равна база? Как dp[i] получается из меньших? Где ответ?

```
// 4 вопроса: что хранит dp[i] (словами)? база? переход? где ответ?

// ЛЕСЕНКА-ПРЫЖКИ (№11): на ступеньку i приходим с i-1 ... i-k
ll jumps(int n, int k) {
    vector<ll> w(n + 1, 0);
    w[0] = 1;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= k && j <= i; j++) w[i] += w[i - j];
    return w[n];
}
```

```
// СУММА S из чисел: внутренний цикл ВНИЗ = каждое число МАКС. ОДИН раз
ll waysDistinct(int S, const vector<int> &nums) {
    vector<ll> dp(S + 1, 0);

    // пустой набор
    dp[0] = 1;
    for (int x : nums)
        for (int s = S; s >= x; s--) dp[s] += dp[s - x];
    return dp[S];
}

// ...внутренний цикл ВВЕРХ = каждое число СКОЛЬКО УГОДНО раз (монеты)
ll waysUnlimited(int S, const vector<int> &coins) {
    vector<ll> dp(S + 1, 0);
    dp[0] = 1;
    for (int x : coins)
        for (int s = x; s <= S; s++) dp[s] += dp[s - x];
    return dp[S];
}
```

```
// РЮКЗАК 0/1: макс. ценность при весе <= W (w ВНИЗ — вещь один раз)
ll knapsack(const vector<int> &wt, const vector<int> &val, int W) {
    vector<ll> best(W + 1, 0);
    for (int i = 0; i < (int)wt.size(); i++)
        for (int w = W; w >= wt[i]; w--)
            best[w] = max(best[w], best[w - wt[i]] + val[i]);
    return best[W];
}

// ПУТИ В ТАБЛИЦЕ: только вправо/вниз, '#' — стена
ll gridPaths(const vector<string> &f) {
    int R = f.size(), C = f[0].size();
    vector<vector<ll>> dp(R, vector<ll>(C, 0));
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++) {
            if (f[i][j] == '#') continue;
            if (i == 0 && j == 0) { dp[i][j] = 1; continue; }
            if (i > 0) dp[i][j] += dp[i - 1][j];
            if (j > 0) dp[i][j] += dp[i][j - 1];
        }
    return dp[R - 1][C - 1];
}

// МАКС. СУММА ПОДОТРЕЗКА: продолжить отрезок или начать заново
ll maxSubarray(const vector<ll> &a) {
    ll cur = 0, best = LLONG_MIN;
    for (ll x : a) { cur = max(x, cur + x); best = max(best, cur); }
    return best;
}

// НВП (строго возрастающая) за n log n
int lis(const vector<int> &a) {
    vector<int> tail;
    for (int x : a) {
        auto it = lower_bound(tail.begin(), tail.end(), x);
```

```
        if (it == tail.end()) tail.push_back(x);
        else *it = x;
    }
    return tail.size();
}
```

11. Строки

```
string toLowerStr(string s) { for (char &c : s) c = tolower(c); return s; }
string toUpperStr(string s) { for (char &c : s) c = toupper(c); return s; }

// строка — только цифры?
bool allDigits(const string &s) {
    if (s.empty()) return false;
    for (char c : s) if (!isdigit(c)) return false;
    return true;
}

// разбить по пробелам
vector<string> splitWords(const string &line) {
    stringstream ss(line);
    vector<string> w;
    string x;
    while (ss >> x) w.push_back(x);
    return w;
}

string replaceAll(string s, const string &from, const string &to) {
    size_t pos = 0;
    while ((pos = s.find(from, pos)) != string::npos) {
        s.replace(pos, from.size(), to);
        pos += to.size();
    }
    return s;
}
```

```
bool isPalindrome(const string &s) {
    return equal(s.begin(), s.end(), s.rbegin());
}

// символ: isdigit isalpha isupper islower isspace;  цифра -> число: c - '0'
// буква k-я: 'a' + k;  номер буквы: c - 'a';  '-' символ, "-" строка
```

Склейка строк и числа из строк

Что нужно	Как
склеить две строки	`a + b` или `a += b` (хотя бы одна сторона — string)
склеить строку и число	`s + to_string(x)` (число сначала в строку)
приклеить символ / повторить символ	`s += 'x'` / `s.append(3, '-')` или `string(3, '-')`
склеить числа: 12 и 34 → 1234	`stoll(to_string(12) + to_string(34))`
склеить список через разделитель	`joinStr(v, "-")`
число → строка	`to_string(x)`
строка → число	`stoi(s)`, `stoll(s)`, `stod(s)` для дроби. Упадёт, если в строке нет числа
символ-цифра → число	`c - '0'` (`'7' - '0' = 7`)
из другой системы счисления	`stoll(s, nullptr, 16)`
цифры строки по одной	`digitsOf(s)`
все числа из текста	`numbersInText("abc12 de-34f 5") = 12 -34 5`
разбить по разделителю	`splitBy("12,7,305", ',') = 12 7 305`
сложить числа длиннее long long	`addBig(a, b)` — раздел 9

```
// ===== СКЛЕЙКА СТРОК =====
// Склеить строки: знак +. Хотя бы с одной стороны должен быть string:
// "abc" + "def" (два голых слова в кавычках) НЕ склеится.
string a = "abc", b = "def";
```

```
// "abcdef"
string c = a + b;

// "abc-def"
string d = a + "-" + b;

// приклеить b к концу a: a = "abcdef"
a += b;

// приклеить один символ
a += 'x';

// приклеить символ '-' три раза
a.append(3, '-');

// число сначала в строку: "n=42"
string s = "n=" + to_string(42);

// склеить числа: 1234
ll glued = stoll(to_string(12) + to_string(34));
```

```
// склеить список строк через разделитель: {"a","b","c"} и "-" дают "a-b-c"
string joinStr(const vector<string> &v, const string &sep) {
    string res;
    for (int i = 0; i < (int)v.size(); i++) {
        // разделитель перед каждым, кроме первого
        if (i > 0) res += sep;
        res += v[i];
    }
    return res;
}

// ===== ЧИСЛА ИЗ СТРОК =====
// stoi("456") -> 456,  stoll("9000000000"),  stod("3.14") -> дробное.
// stoi("12abc") даёт 12 (читает, пока цифры). stoi("abc") и слишком большое число
// ВЫБРАСЫВАЮТ ОШИБКУ (программа упадёт): сначала проверь isdigit.
```

```
// строка -> число вручную, по цифрам: "4057" -> 4057
ll strToNum(const string &s) {
    ll r = 0;

    // сдвинуть влево на разряд и добавить цифру
    for (char c : s) r = r * 10 + (c - '0');
    return r;
}
```

```
// цифры строки как числа: "4057" -> {4, 0, 5, 7}
vector<int> digitsOf(const string &s) {
    vector<int> d;
    for (char c : s)
        if (isdigit((unsigned char)c)) d.push_back(c - '0');
    return d;
}

// ВСЕ числа из текста: "abc12 de-34f 5" -> {12, -34, 5}
vector<ll> numbersInText(const string &s) {
    vector<ll> res;
    int n = s.size();
    for (int i = 0; i < n;) {
        if (isdigit((unsigned char)s[i])) {
            // минус прямо перед цифрой
            bool neg = (i > 0 && s[i - 1] == '-');
            ll x = 0;
            while (i < n && isdigit((unsigned char)s[i])) {
                x = x * 10 + (s[i] - '0');
                i++;
            }
            res.push_back(neg ? -x : x);
        } else {
            i++;
        }
    }
    return res;
```

```
}
```

```
// разбить по разделителю: "12,7,305" и ',' дают {"12", "7", "305"}
vector<string> splitBy(const string &s, char sep) {
    vector<string> res;
    stringstream ss(s);
    string item;
    while (getline(ss, item, sep)) res.push_back(item);
    return res;
}
// сложить два ОЧЕНЬ больших числа, записанных строками — addBig (раздел 9)
```

12. Геометрия

Пока можно, считай в целых (long long), без sqrt и double. Расстояния сравнивай через квадраты.

Математика	В C++ (<cmath>)	Внимание
расстояние между (x1,y1) и (x2,y2)	`hypot(x2 - x1, y2 - y1)`	или sqrt(dx*dx + dy*dy)
число π	`acos(-1.0)`	M_PI есть не везде
sin α, cos α	`sin(a), cos(a)`	угол в РАДИАНАХ
градусы → радианы	`deg * PI / 180`	
угол вектора (x, y)	`atan2(y, x)`	радианы, от −π до π
|x| для дроби	`fabs(x)`	
x^2	`x * x`	не x ^ 2
площадь круга	`PI * r * r`	
середина отрезка	`(x1 + x2) / 2.0`	2.0, чтобы не отбросить дробь
a равно b (дроби)	`fabs(a - b) < 1e-9`	не a == b
вывод с k знаками	`cout << fixed << setprecision(k) << x`	<iomanip>

```
// Считай в ЦЕЛЫХ (ll). Расстояния сравнивай КВАДРАТАМИ, без sqrt.
struct Pt { ll x, y; };
```

```
ll dist2(Pt a, Pt b) {
    return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
}
// >0: b левее луча o->a (против часовой); <0: правее; 0: на одной прямой
// |cross| = УДВОЕННАЯ площадь треугольника
ll cross(Pt o, Pt a, Pt b) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}
ll area2(Pt a, Pt b, Pt c) { return llabs(cross(a, b, c)); }
```

```
// многоугольник, вершины по порядку
ll polyArea2(const vector<Pt> &p) {
    ll s = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        s += p[i].x * p[j].y - p[j].x * p[i].y;
    }
    // удвоенная площадь
    return llabs(s);
}
```

```
// внутри или на границе (№12)
bool inTriangle(Pt a, Pt b, Pt c, Pt p) {
    return area2(a, b, p) + area2(b, c, p) + area2(c, a, p) == area2(a, b, c);
}
```

```
// p лежит на отрезке ab
bool onSegment(Pt a, Pt b, Pt p) {
    return cross(a, b, p) == 0 &&
           min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
int sgn(ll v) { return (v > 0) - (v < 0); }

// отрезки ab и cd пересекаются?
bool segIntersect(Pt a, Pt b, Pt c, Pt d) {
    int d1 = sgn(cross(a, b, c)), d2 = sgn(cross(a, b, d));
    int d3 = sgn(cross(c, d, a)), d4 = sgn(cross(c, d, b));
    if (d1 * d2 < 0 && d3 * d4 < 0) return true;
    return onSegment(a, b, c) || onSegment(a, b, d) ||
           onSegment(c, d, a) || onSegment(c, d, b);
}
```

```
// две окружности: число общих точек (3 = совпадают). Сравниваем КВАДРАТЫ (№26)
int circles(Pt c1, ll r1, Pt c2, ll r2) {
    ll d2 = dist2(c1, c2), sum = r1 + r2, diff = llabs(r1 - r2);
    if (d2 == 0 && r1 == r2) return 3;

    // далеко ИЛИ внутри
    if (d2 > sum * sum || d2 < diff * diff) return 0;

    // касаются
    if (d2 == sum * sum || d2 == diff * diff) return 1;
    return 2;
}
// double: сравнивай fabs(a - b) < 1e-9;  PI = acos(-1.0);  вывод: fixed << setprecision(k)
```

13. Графы

```
// Чтение (вершины 1..n):  vector<vector<int>> adj(n + 1);
// for m рёбер: cin >> u >> v; adj[u].push_back(v); adj[v].push_back(u);
// Матрица смежности (№15): g[i][j] == 1 -> ребро; рёбер = (сумма единиц) / 2

// шаги от s, -1 = недостижима
vector<int> bfs(const vector<vector<int>> &adj, int s) {
    vector<int> d(adj.size(), -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int to : adj[v])
            if (d[to] == -1) { d[to] = d[v] + 1; q.push(to); }
    }
    return d;
}
```

```
// DFS: компоненты связности
vector<vector<int>> G;
vector<bool> used;
void dfs(int v) {
    used[v] = true;
    for (int to : G[v]) if (!used[to]) dfs(to);
}
// comps = 0; for v: if (!used[v]) { dfs(v); comps++; }
```

```
int gridBFS(const vector<string> &f, int sr, int sc, int er, int ec) {
    int R = f.size(), C = f[0].size();
    int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    vector<vector<int>> d(R, vector<int>(C, -1));
    queue<pair<int, int>> q;
    d[sr][sc] = 0;
    q.push({sr, sc});
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            // за полем
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            // стена/были
            if (f[nr][nc] == '#' || d[nr][nc] != -1) continue;
            d[nr][nc] = d[r][c] + 1;
            q.push({nr, nc});
        }
    }
    // -1, если не дойти
    return d[er][ec];
}
```

```
// веса >= 0
vector<ll> dijkstra(const vector<vector<pair<int, int>>> &g, int s) {
    vector<ll> D(g.size(), LLONG_MAX);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
    D[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [dv, v] = pq.top(); pq.pop();
        if (dv > D[v]) continue;
        for (auto [to, w] : g[v])
            if (D[v] + w < D[to]) { D[to] = D[v] + w; pq.push({D[to], to}); }
    }
    return D;
}
// группы: объединить / вместе ли
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

14. Перебор и жадность

Перебор: пробуем все варианты и выбираем подходящий. **Битовая маска:** число mask от 0 до $2^n - 1$. Если бит i равен 1, элемент i «берём». **(1 << i)** — число с единственной единицей на месте i; **mask & (1 << i)** не ноль, если бит i включён; **1 << n** = 2^n — сколько всего масок. Пример для n = 3: mask = 5 = 101 → берём элементы 0 и 2.

Жадность: на каждом шаге берём самое выгодное. Часто сначала sort, потом проход по порядку. **Перестановки:** sort, затем do { ... } while (next_permutation(p.begin(), p.end())) — перебирает все порядки.

```
// ВСЕ ПОДМНОЖЕСТВА (n <= 20): бит i в mask = берём a[i]
int countSubsetsWithSum(const vector<int> &a, int target) {
    int n = a.size(), cnt = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int s = 0;
        for (int i = 0; i < n; i++) if (mask & (1 << i)) s += a[i];
        if (s == target) cnt++;
    }
    return cnt;
}
// ВСЕ ПЕРЕСТАНОВКИ (n <= 10): сначала sort!
// sort(p.begin(), p.end());
// do { ...проверить p... } while (next_permutation(p.begin(), p.end()));
```

```
// ЖАДНОСТЬ "большое с большим" (№685): сортируем оба и умножаем попарно
ll maxPairSum(vector<ll> a, vector<ll> b) {
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());
    ll s = 0;
    for (int i = 0; i < (int)a.size(); i++) s += a[i] * b[i];
    return s;
}
```

15. Комбинаторика

Что считаем	Формула	C++
перестановки n разных	n!	`fact(n)`
k из n, порядок ВАЖЕН	n! / (n−k)!	`nPk(n, k)`
k из n, порядок НЕ важен	C(n,k) = n! / (k!·(n−k)!)	`nCk(n, k)`
с повторами, порядок важен	n^k	`powll(n, k)`
с повторами, порядок не важен	C(n+k−1, k)	`nCk(n + k - 1, k)`
перестановки с одинаковыми	n! / (a!·b!·…)	`fact(n) / (fact(a) * fact(b))`
все подмножества	2^n	`1LL << n`
числа Каталана	C(2n,n) / (n+1)	`catalan(n)`
1 + 2 + … + n	n(n+1) / 2	`n * (n + 1) / 2`
арифметическая прогрессия	$(a_1 + a_n)\cdot n / 2$	`(a1 + an) * n / 2`
«и» / «или»	умножаем / складываем	`x * y / x + y`

```
// Формула -> C++. Всё в long long. n! помещается в long long только до n = 20.
ll fact(int n) {
    ll r = 1;
    // 1 * 2 * 3 * ... * n
    for (int i = 2; i <= n; i++) r *= i;
    return r;
}
```

```
// размещения: n! / (n-k)!   (выбрать k из n, порядок важен)
ll nPk(int n, int k) {
    ll r = 1;
    // n * (n-1) * ... (k штук)
    for (int i = 0; i < k; i++) r *= n - i;
    return r;
}
```

```
// сочетания: C(n,k) = n! / (k! * (n-k)!)   (порядок не важен)
ll nCk(int n, int k) {
    // C(n,k) = C(n,n-k): меньше шагов
    if (k > n - k) k = n - k;
    ll r = 1;
    for (int i = 1; i <= k; i++)
```

```
        // после каждого шага делится нацело
        r = r * (n - k + i) / i;
    return r;
}
```

```
// a в степени k (циклом)
ll powll(ll a, int k) {
    ll r = 1;
    while (k--) r *= a;
    return r;
}
```

```
// 1 + 2 + ... + n
ll sumTo(ll n)   { return n * (n + 1) / 2; }

// 2 в степени n
ll pow2(int n)   { return 1LL << n; }
ll catalan(int n) { return nCk(2 * n, n) / (n + 1); }
```

```
// C(n, k) треугольником Паскаля: C[n][k] = C[n-1][k-1] + C[n-1][k]   (n <= 60 без модуля)
vector<vector<ll>> pascal(int N) {
    vector<vector<ll>> C(N + 1, vector<ll>(N + 1, 0));
    for (int n = 0; n <= N; n++) {
        C[n][0] = 1;
        for (int k = 1; k <= n; k++) C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
    }
    return C;
}

// C(n, k) по модулю MOD для больших n (до ~10^6): факториалы + обратные
struct Comb {
    vector<ll> f, inv;
    Comb(int n) : f(n + 1), inv(n + 1) {
        f[0] = 1;
        for (int i = 1; i <= n; i++) f[i] = f[i - 1] * i % MOD;
        inv[n] = binpow(f[n], MOD - 2, MOD);
        for (int i = n; i > 0; i--) inv[i - 1] = inv[i] * i % MOD;
    }
    ll C(int n, int k) { return (k < 0 || k > n) ? 0 : f[n] * inv[k] % MOD * inv[n - k] % MOD; }
};
```

```
// ПЕРЕБОР РЕКУРСИЕЙ: все наборы из k чисел 1..n по возрастанию
vector<int> pick;
void genCombos(int start, int n, int k, vector<vector<int>> &out) {
    if ((int)pick.size() == k) { out.push_back(pick); return; }
    for (int x = start; x <= n; x++) {
        // взяли x
        pick.push_back(x);
        // добираем остальное
        genCombos(x + 1, n, k, out);
        // ОТКАТ: убрали x, пробуем следующий
        pick.pop_back();
    }
}
```

16. Строки-2: префикс-функция, Z-функция, хеш

Префикс-функция: p[i] — длина самого длинного куска, который одновременно является началом строки и заканчивается в позиции i. Пример: для «abacaba» p = 0 0 1 0 1 2 3 (на конце «aba» — и начало, и конец). Зачем: найти все вхождения слова в тексте (склеить «слово # текст»), найти период строки: если n % (n − p[n−1]) == 0, строка состоит из повторов куска длиной n − p[n−1]. **Z-функция:** z[i] — длина общего начала строки и её куска с позиции i. **Хеш:** число, по которому равные куски строки легко сравнить за один шаг.

```
// ПРЕФИКС-ФУНКЦИЯ: p[i] = длина самого длинного префикса s, который заканчивается в i
vector<int> prefixFunc(const string &s) {
    int n = s.size();
    vector<int> p(n, 0);
    for (int i = 1; i < n; i++) {
        int k = p[i - 1];
        while (k > 0 && s[i] != s[k]) k = p[k - 1];
        if (s[i] == s[k]) k++;
        p[i] = k;
    }
    return p;
}
// ВСЕ ВХОЖДЕНИЯ pat в text (КМП): склеиваем pat + '#' + text ('#' нет в строках)
vector<int> findAll(const string &text, const string &pat) {
    vector<int> p = prefixFunc(pat + '#' + text), res;
    int m = pat.size();
    for (int i = 0; i < (int)p.size(); i++)
        // начало вхождения в text
        if (p[i] == m) res.push_back(i - 2 * m);
    return res;
}
```

```
// Z-ФУНКЦИЯ: z[i] = длина общего префикса s и s[i..]
vector<int> zFunc(const string &s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}
```

```
// ПОЛИНОМИАЛЬНЫЙ ХЕШ: сравнить любые две подстроки за O(1)
struct Hash {
    static const ll M = 1000000007, B = 131;
    vector<ll> h, pw;
    Hash(const string &s) : h(s.size() + 1, 0), pw(s.size() + 1, 1) {
        for (int i = 0; i < (int)s.size(); i++) {
            h[i + 1] = (h[i] * B + s[i]) % M;
            pw[i + 1] = pw[i] * B % M;
        }
    }
    // хеш s[l..r), r НЕ включая
    ll get(int l, int r) {
        return ((h[r] - h[l] * pw[r - l]) % M + M) % M;
    }
};
// подстроки равны, если хеши равны (ложное совпадение — очень редко)
```

17. Структуры данных

Ситуация	Что брать
массив НЕ меняется, сумма на отрезке	префиксные суммы (раздел 8)
массив НЕ меняется, мин/макс на отрезке	разреженная таблица SparseMin
элементы меняются + сумма	Фенвик (короче всего писать)
элементы меняются + мин/макс/сумма	дерево отрезков SegTree
не помнишь дерево — нужно что-то проще	sqrt-декомпозиция (медленнее, но работает)

```
// ДЕРЕВО ФЕНВИКА: a[i] += x и сумма на отрезке — за log n
struct Fenwick {
    int n;
    vector<ll> t;
    Fenwick(int n) : n(n), t(n + 1, 0) {}

    // a[i] += x
    void add(int i, ll x) { for (i++; i <= n; i += i & -i) t[i] += x; }

    // a[0..i]
    ll pref(int i) { ll s = 0; for (i++; i > 0; i -= i & -i) s += t[i]; return s; }

    // a[l..r]
    ll sum(int l, int r) { return pref(r) - (l > 0 ? pref(l - 1) : 0); }
};
```

```
// СТАТИЧЕСКИЙ RMQ (разреженная таблица): минимум на отрезке за O(1), массив НЕ меняется
struct SparseMin {
    vector<vector<ll>> t;

    // lg[x] = целая часть log2(x)
    vector<int> lg;
    SparseMin(const vector<ll> &a) {
        int n = a.size(), K = 1;
        while ((1 << K) <= n) K++;
        lg.assign(n + 2, 0);
        for (int i = 2; i <= n + 1; i++) lg[i] = lg[i / 2] + 1;
        t.assign(K, vector<ll>(n));
        t[0] = a;
        for (int k = 1; k < K; k++)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = min(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
    }

    // минимум a[l..r] включительно
    ll get(int l, int r) {
        int k = lg[r - l + 1];
        return min(t[k][l], t[k][r - (1 << k) + 1]);
    }
};
```

```
// ДЕРЕВО ОТРЕЗКОВ: a[i] = x и минимум на отрезке — за log n
// для СУММЫ: замени все min(...) на сложение, а NEUTRAL на 0
struct SegTree {
    int n;
    vector<ll> t;
    const ll NEUTRAL = LLONG_MAX;
    SegTree(const vector<ll> &a) : n(a.size()), t(2 * a.size()) {
        for (int i = 0; i < n; i++) t[n + i] = a[i];
        for (int i = n - 1; i > 0; i--) t[i] = min(t[2 * i], t[2 * i + 1]);
    }

    // a[i] = x
    void set(int i, ll x) {
        for (t[i += n] = x; i > 1; i /= 2) t[i / 2] = min(t[i], t[i ^ 1]);
    }

    // минимум a[l..r] включительно
    ll get(int l, int r) {
        ll res = NEUTRAL;
        for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
            if (l & 1) res = min(res, t[l++]);
            if (r & 1) res = min(res, t[--r]);
        }
        return res;
    }
};
```

```
// SQRT-ДЕКОМПОЗИЦИЯ: блоки по ~sqrt(n), храним сумму каждого блока
struct SqrtSum {
    int B;
    vector<ll> a, blk;
    SqrtSum(const vector<ll> &v)
        : B(max(1, (int)sqrt((double)v.size()))), a(v), blk(v.size() / B + 1, 0) {
        for (int i = 0; i < (int)a.size(); i++) blk[i / B] += a[i];
    }
    void set(int i, ll x) { blk[i / B] += x - a[i]; a[i] = x; }

    // a[l..r] включительно
    ll sum(int l, int r) {
        ll s = 0;

        // хвост слева
        while (l <= r && l % B != 0) s += a[l++];

        // целые блоки
        while (l + B - 1 <= r) { s += blk[l / B]; l += B; }

        // хвост справа
        while (l <= r) s += a[l++];
        return s;
    }
};
```

18. Графы-2

Задача	Алгоритм
кратчайший путь, все рёбра одинаковые	BFS (раздел 13)
кратчайший путь, веса ≥ 0	Дейкстра (раздел 13)
есть отрицательные веса / найти отрицательный цикл	Форд-Беллман
расстояния между ВСЕМИ парами, n ≤ 400	Флойд
порядок дел с зависимостями, есть ли цикл	топосорт
можно ли разбить на две группы без рёбер внутри	двудольность
соединить все вершины минимальной суммой	Краскал + DSU
группы «каждый достижим из каждого» (стрелки)	конденсация
пройти каждое ребро ровно один раз	Эйлеров цикл
максимум пар «левый — правый»	паросочетание (Кун)

```
// ТОПОСОРТ (Кан): порядок вершин 0..n-1, где все стрелки идут вперёд.
// Пустой ответ = в графе есть цикл.
vector<int> topoSort(const vector<vector<int>> &g) {
    int n = g.size();
    vector<int> indeg(n, 0), order;
    for (int v = 0; v < n; v++) for (int to : g[v]) indeg[to]++;
    queue<int> q;
    for (int v = 0; v < n; v++) if (indeg[v] == 0) q.push(v);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        order.push_back(v);
        for (int to : g[v]) if (--indeg[to] == 0) q.push(to);
    }
    if ((int)order.size() < n) return {};
    return order;
}

// ДВУДОЛЬНОСТЬ: покрасить в 2 цвета так, чтобы каждое ребро соединяло разные цвета
bool isBipartite(const vector<vector<int>> &g) {
    int n = g.size();
    vector<int> col(n, -1);
    for (int s = 0; s < n; s++) {
        if (col[s] != -1) continue;
        col[s] = 0;
        queue<int> q;
        q.push(s);
```

```
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int to : g[v]) {
                if (col[to] == -1) { col[to] = col[v] ^ 1; q.push(to); }
                else if (col[to] == col[v]) return false;
            }
        }
    }
    return true;
}
```

```
// ФЛОЙД: кратчайшие пути между ВСЕМИ парами (n <= 400). d[i][j] = вес или INF, d[i][i] = 0
const ll INF = LLONG_MAX / 4;
void floyd(vector<vector<ll>> &d) {
    int n = d.size();
    // k — САМЫЙ ВНЕШНИЙ цикл!
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

```
// ФОРД-БЕЛЛМАН: пути из s, веса МОГУТ быть отрицательными.
// false = есть отрицательный цикл, достижимый из s
struct Edge { int u, v; ll w; };
bool bellmanFord(int n, const vector<Edge> &edges, int s, vector<ll> &d) {
    d.assign(n, INF);
    d[s] = 0;
    // n-1 раз улучшаем все рёбра
    for (int it = 0; it < n - 1; it++)
        for (auto &e : edges)
            if (d[e.u] < INF && d[e.u] + e.w < d[e.v]) d[e.v] = d[e.u] + e.w;
    // ещё улучшается -> отрицательный цикл
    for (auto &e : edges)
        if (d[e.u] < INF && d[e.u] + e.w < d[e.v]) return false;
    return true;
}
```

```
// МИНИМАЛЬНЫЙ КАРКАС (Краскал): рёбра по весу, берём ребро, если оно не создаёт цикл
// для неориентированного графа
ll kruskal(int n, vector<Edge> edges) {
    sort(edges.begin(), edges.end(), [](const Edge &a, const Edge &b) { return a.w < b.w; });
    DSU dsu(n);
    ll total = 0;
    for (auto &e : edges)
        if (dsu.unite(e.u, e.v)) total += e.w;
    return total;
}
```

```
// КОНДЕНСАЦИЯ (Косарайю): comp[v] — номер компоненты сильной связности
vector<int> sccOrder, sccComp;
vector<bool> sccUsed;
void sccDfs1(const vector<vector<int>> &g, int v) {
    sccUsed[v] = true;
    for (int to : g[v]) if (!sccUsed[to]) sccDfs1(g, to);
    sccOrder.push_back(v);
}
void sccDfs2(const vector<vector<int>> &gr, int v, int c) {
    sccComp[v] = c;
    for (int to : gr[v]) if (sccComp[to] == -1) sccDfs2(gr, to, c);
}
// вернёт число компонент
int scc(const vector<vector<int>> &g) {
    int n = g.size();
```

```
    // граф с развёрнутыми стрелками
    vector<vector<int>> gr(n);
    for (int v = 0; v < n; v++) for (int to : g[v]) gr[to].push_back(v);
    sccOrder.clear();
    sccUsed.assign(n, false);
    sccComp.assign(n, -1);
    for (int v = 0; v < n; v++) if (!sccUsed[v]) sccDfs1(g, v);
    int c = 0;
    for (int i = n - 1; i >= 0; i--)
        if (sccComp[sccOrder[i]] == -1) sccDfs2(gr, sccOrder[i], c++);
    return c;
}
```

```
// ЭЙЛЕРОВ ЦИКЛ (неориентированный): пройти КАЖДОЕ ребро ровно один раз.
// Есть, если рёбра связаны и у всех вершин чётная степень.
vector<int> eulerCycle(int n, const vector<pair<int, int>> &edges, int start) {
    // {сосед, номер ребра}
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)edges.size(); i++) {
        g[edges[i].first].push_back({edges[i].second, i});
        g[edges[i].second].push_back({edges[i].first, i});
    }
    vector<bool> usedE(edges.size(), false);
    vector<int> ptr(n, 0), st = {start}, path;
    while (!st.empty()) {
        int v = st.back();
        while (ptr[v] < (int)g[v].size() && usedE[g[v][ptr[v]].second]) ptr[v]++;
        if (ptr[v] == (int)g[v].size()) { path.push_back(v); st.pop_back(); }
        else {
            auto [to, id] = g[v][ptr[v]];
            usedE[id] = true;
            st.push_back(to);
        }
    }
    // вершины по порядку, рёбер + 1 штук
    return path;
}
```

```
// ПАРОСОЧЕТАНИЕ (Кун): g[левая] = список правых (0..n2-1). Максимум пар
// matchR[правая] = левая или -1
vector<int> matchR;
vector<bool> kuhnUsed;
bool kuhn(const vector<vector<int>> &g, int v) {
    if (kuhnUsed[v]) return false;
    kuhnUsed[v] = true;
    for (int to : g[v])
        if (matchR[to] == -1 || kuhn(g, matchR[to])) { matchR[to] = v; return true; }
    return false;
}
int maxMatching(const vector<vector<int>> &g, int n2) {
    matchR.assign(n2, -1);
    int res = 0;
    for (int v = 0; v < (int)g.size(); v++) {
        kuhnUsed.assign(g.size(), false);
        if (kuhn(g, v)) res++;
    }
    return res;
}
```

19. Ещё динамика и деление длинного числа

```
// НАИБОЛЬШАЯ ОБЩАЯ ПОДПОСЛЕДОВАТЕЛЬНОСТЬ: dp[i][j] — для первых i букв a и j букв b
int lcs(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i - 1] == b[j - 1]) ? dp[i - 1][j - 1] + 1
                                              : max(dp[i - 1][j], dp[i][j - 1]);
    return dp[n][m];
}

// РЕДАКЦИОННОЕ РАССТОЯНИЕ: минимум вставок, удалений и замен, чтобы из a получить b
int editDist(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = min({dp[i - 1][j] + 1, dp[i][j - 1] + 1,
                            dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])});
    return dp[n][m];
}

// МИНИМУМ МОНЕТ на сумму S (монеты сколько угодно раз); -1 — нельзя набрать
int minCoins(int S, const vector<int> &coins) {
    vector<int> dp(S + 1, INT_MAX);
    dp[0] = 0;
    for (int s = 1; s <= S; s++)
        for (int c : coins)
            if (c <= s && dp[s - c] != INT_MAX) dp[s] = min(dp[s], dp[s - c] + 1);
    return dp[S] == INT_MAX ? -1 : dp[S];
}

// ДЛИННОЕ / k (k <= 10^9): возвращает частное, остаток кладёт в rem
string divSmall(const string &a, ll k, ll &rem) {
    string res;
    rem = 0;
    for (char c : a) {
        rem = rem * 10 + (c - '0');
        res += char('0' + rem / k);
        rem %= k;
    }
    size_t p = res.find_first_not_of('0');
    return p == string::npos ? "0" : res.substr(p);
}
```

20. Геометрия-2

```
// ВЫПУКЛАЯ ОБОЛОЧКА (Эндрю): вершины против часовой, без точек на сторонах
vector<Pt> convexHull(vector<Pt> p) {
    sort(p.begin(), p.end(), [](Pt a, Pt b) { return a.x < b.x || (a.x == b.x && a.y < b.y); });
    p.erase(unique(p.begin(), p.end(), [](Pt a, Pt b) { return a.x == b.x && a.y == b.y; }),
            p.end());
    int n = p.size();
    if (n < 3) return p;
    vector<Pt> h(2 * n);
    int k = 0;
    // нижняя часть
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(h[k - 2], h[k - 1], p[i]) <= 0) k--;
        h[k++] = p[i];
    }
    // верхняя часть
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(h[k - 2], h[k - 1], p[i]) <= 0) k--;
        h[k++] = p[i];
    }
    h.resize(k - 1);
    return h;
}
```

```
// ТОЧКА В МНОГОУГОЛЬНИКЕ (любом, вершины по порядку): 0 — снаружи, 1 — внутри, 2 — на границе
// Луч вправо от q: считаем, сколько сторон он пересекает (нечётно = внутри)
int inPolygon(const vector<Pt> &p, Pt q) {
    int n = p.size();
    bool in = false;
    for (int i = 0; i < n; i++) {
        Pt a = p[i], b = p[(i + 1) % n];
        if (onSegment(a, b, q)) return 2;
        // сторона пересекает строку y = q.y
        if ((a.y > q.y) != (b.y > q.y)) {
            // пересечение правее q? (в целых)
            ll c = cross(a, b, q);
            if ((c > 0) == (b.y > a.y)) in = !in;
        }
    }
    return in ? 1 : 0;
}
```

21. Python: разрешённые библиотеки и функции

Python разрешён со стандартными библиотеками из поставки. **numpy и pandas — нет.** Целые числа любой длины, переполнения нет. Скорость примерно 10^7 простых операций в секунду: на больших данных (10^6 и больше) лучше C++.

Библиотека	Что даёт	Функции
встроенные	без import	sorted, sum, min, max, abs, pow(a, b, m), divmod, enumerate, zip, map, range, len, ord, chr
math	числа	gcd, lcm (3.9+), isqrt, sqrt, hypot, atan2, pi, floor, ceil, factorial, comb, perm (3.8+)
itertools	перебор	permutations, combinations, product, accumulate (префиксные суммы)
bisect	бинпоиск	bisect_left (= lower_bound), bisect_right (= upper_bound)
heapq	куча	heappush, heappop (наверху минимум; максимум — класть с минусом)
collections	структуры	Counter, defaultdict, deque (append, popleft)
functools	память рекурсии	lru_cache(maxsize=None) над функцией
sys	ввод и лимиты	stdin.readline (быстрый ввод), setrecursionlimit, set_int_max_str_digits(0)
re	шаблоны в строках	re.findall(r'-?\d+', s) — все числа из текста; re.split — разбить

C++ → Python (что чем заменить)

C++	Python	C++	Python
vector	list	long long	int (без переполнения)
pair	кортеж (a, b)	to_string / stoi	str(x) / int(s)
sort(v)	v.sort() / sorted(v)	lower_bound	bisect_left(v, x)
priority_queue	heapq (минимум)	upper_bound	bisect_right(v, x)
map	dict	next_permutation	permutations(v)
set	set	accumulate	sum(v) / accumulate(v)
queue	deque	popcount	bin(x).count("1")
gcd, lcm	math.gcd, math.lcm	a + b (строки)	то же самое

Подводные камни Python

• **input()** возвращает строку: int(input()), list(map(int, input().split())).

• **Строки и числа:** склейка a + b, "-".join(список), f"{a}{b}"; число → строка str(x), строка → число int(s), float(s); если в строке не число — ValueError; разбить: s.split(",").

• **Таблица:** [[0] * m for _ in range(n)], а не [[0] * m] * n (во втором все строки одна и та же).

• **//** — деление нацело вниз (-7 // 2 = -4), **/** — всегда дробь. round(2.5) = 2.

• Большой ввод: input = sys.stdin.readline (строку чистить .strip()). Глубокая рекурсия: sys.setrecursionlimit(10 ** 6).

• Строки неизменяемы: s[0] = 'a' нельзя. Больше 4300 цифр при str() / int(): sys.set_int_max_str_digits(0).